International Software Engineering Program
International College
King Mongkut's Institute of Technology Ladkrabang

13016214 Software Engineering Principle

Final Examination

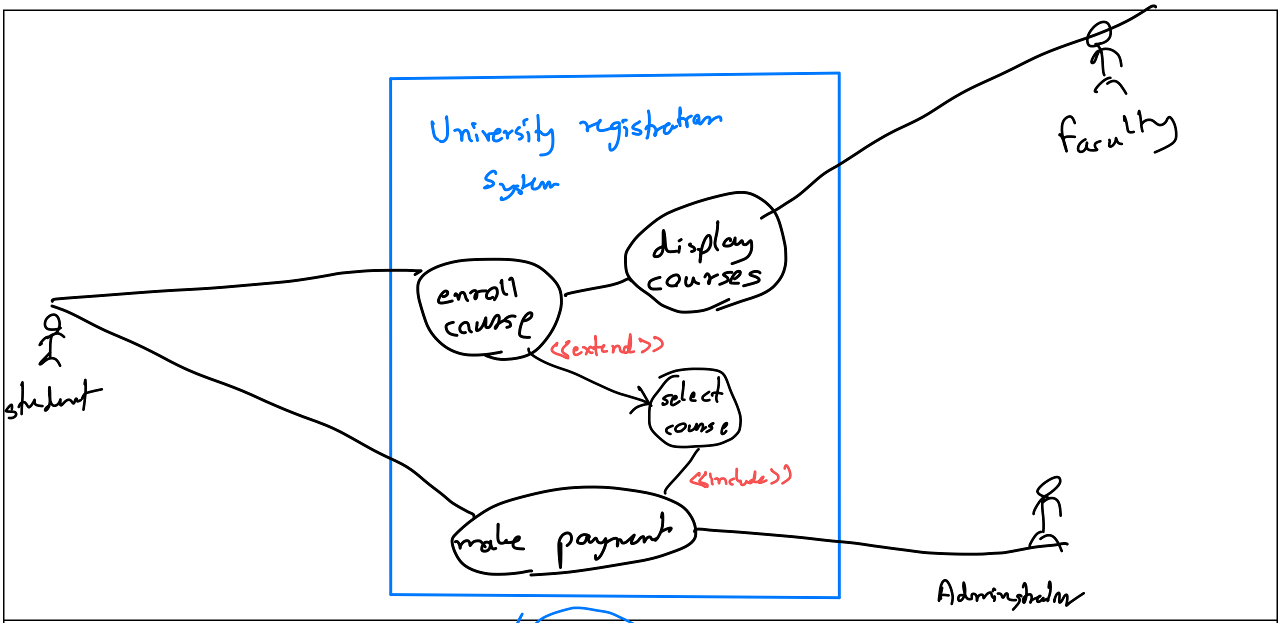
Tue 26th May 2020, 2:00 – 4:30 pm.

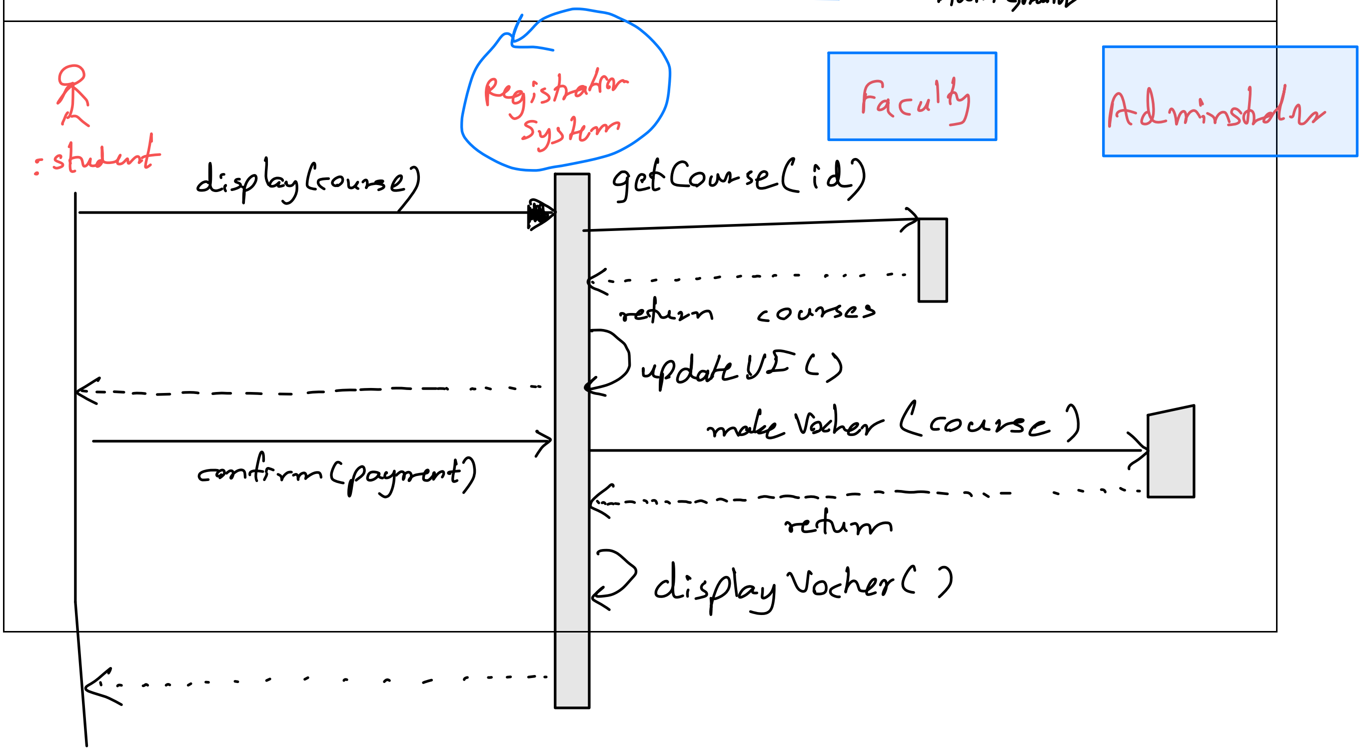
Instruction

1. This is an **opened book** exam.
2. There are **5** questions, please **answer all** of them.

Questions:

1. (5 marks) Draw **a use case diagram** of a university registration system. Then draw **one sequence diagram** of a use case **"enroll for a course"** for *its basic course of events*.



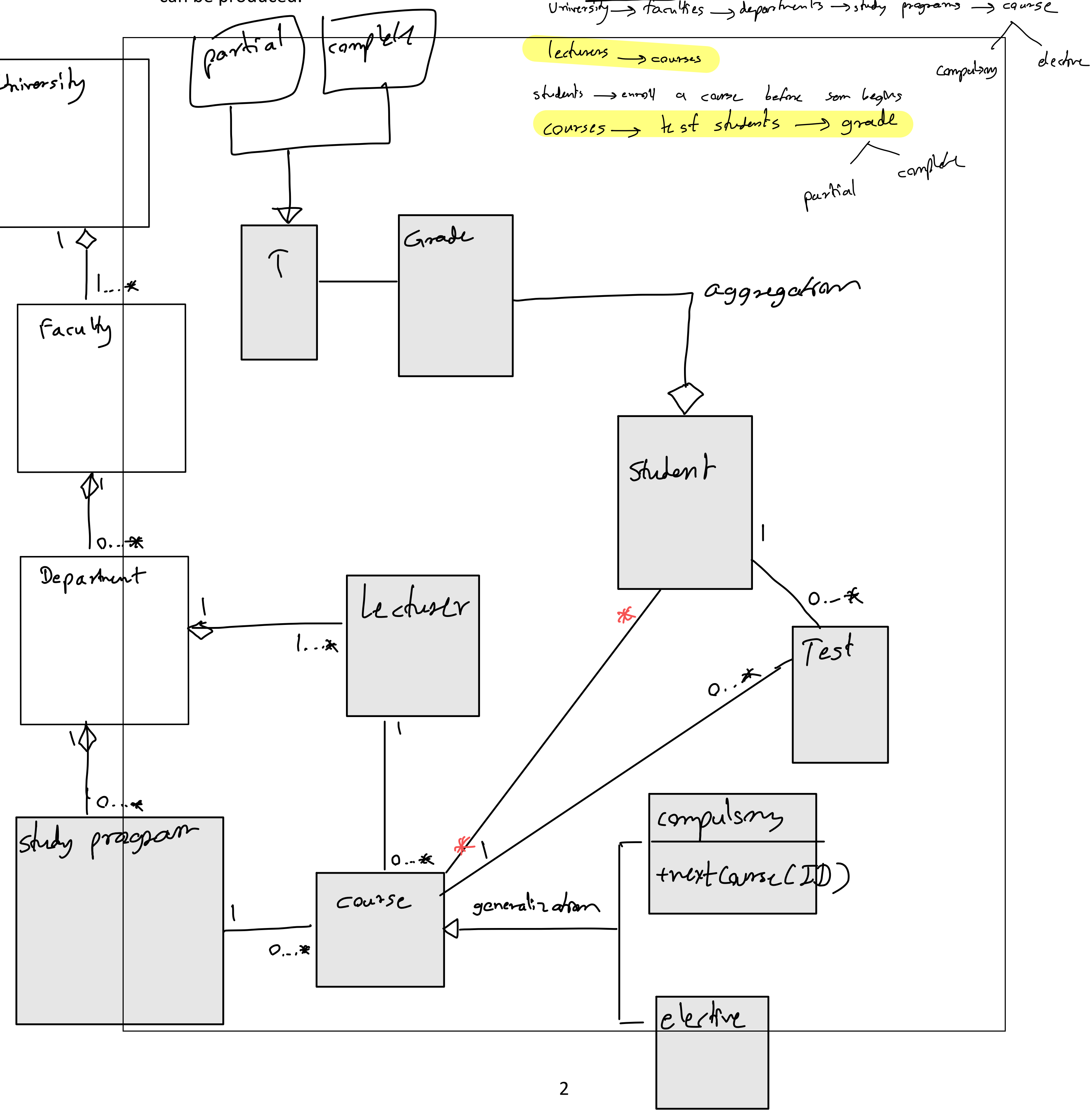


2. (5 marks) Draw class diagrams to model, at the application layer, the university registration system in **Question 1**. In each class of the diagrams, please identify **necessary class attributes and methods**.

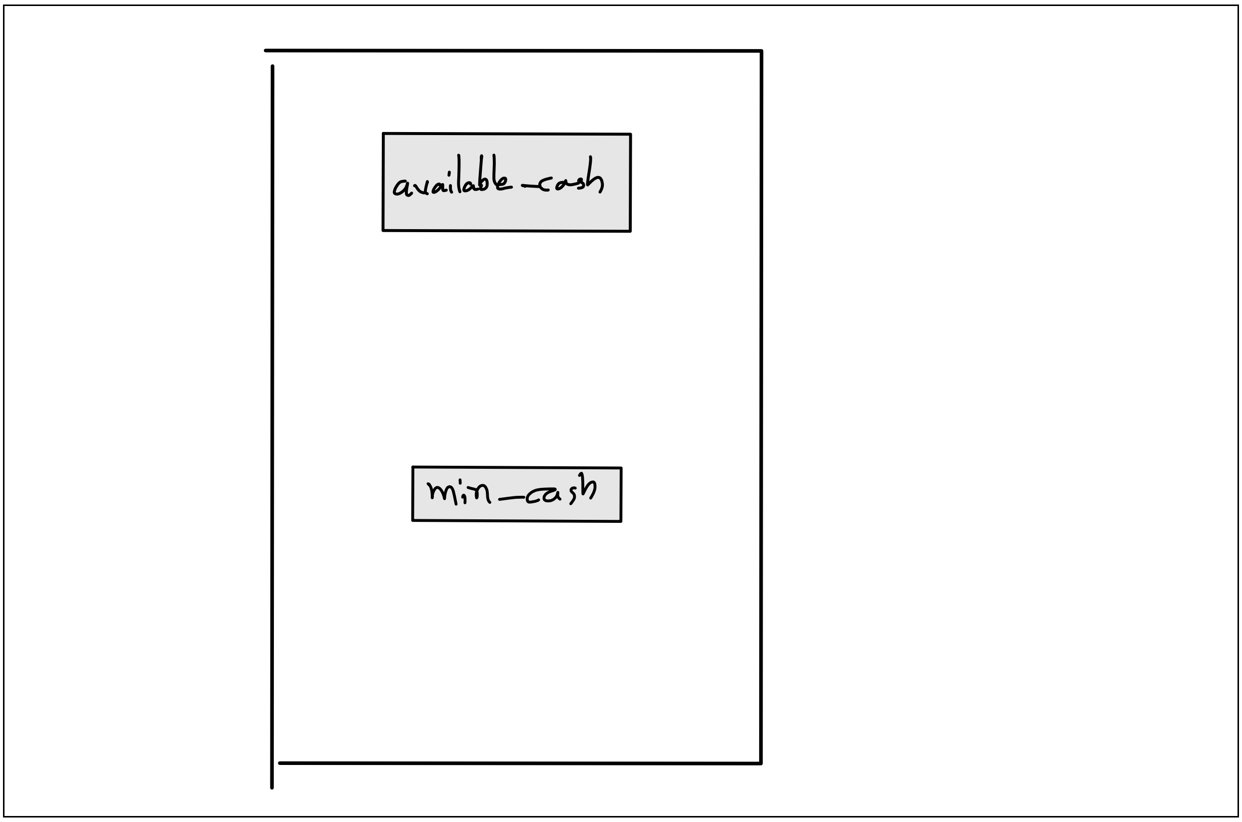
The system description is as follows:

A university consists of many faculties, each of which has many departments, and each department offers several study programs. Each study program contains courses, either compulsory or elective ones.

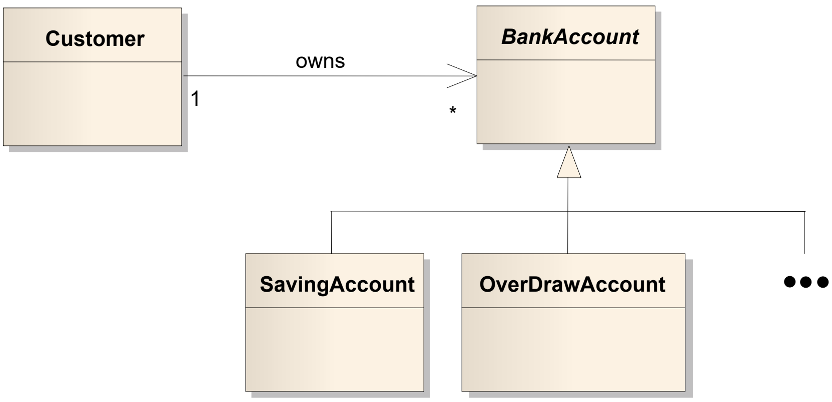
In each semester, lecturers offer courses for students to enroll before a semester begins and the students will be assessed, while taking the ~~course, by many tests. At the~~ end of each semester the students are evaluated by grading, as a result a partial or complete transcript for each student can be produced.



3. (5 marks) Draw **a statechart diagram** describing a life-cycle of an object of a class EWallet. The class EWallet has at least attributes: available_cash and min_cash. EWallet has at least methods: deposit, withdraw, and display_cash. Whenever the available_cash in an EWallet is less than min_cash, an object of EWallet will alert to let the user know the available_cash in it is too low and needs to be added to exceed min_cash. When an object of EWallet is created, its available_cash will always be set to 0.



4. (5 marks) Please write down a Python program according to the following class diagram design below. Define a **BankAccount** class which has the following **properties** and **methods**:



- **Properties**
  - **bank_name**: the name of the bank who creates the account
  - **acc_name**: the name and surname of the account owner
  - **acc_id**: the account id

- **balance**: the current balance of the account
- **Methods**
  - **get_balance()**: get the current balance of a bank account
  - **deposit(money,person,date)**: deposit **money** by **person** to the account on **date**
  - **withdraw(money,person,date)**: withdraw **money** by **person** from the account on **date**

Define **SavingAccount** as a subclass of **BankAccount**, and define necessary properties and methods for this sub-class.

Define **OverDrawnAccount** as another sub-class of **BankAccount**. An overdrawn account is different from a saving account very little, in that, a saving account cannot have a negative balance whilst an overdrawn account can have a negative balance which does not exceed **the over drawn limit**. Please define necessary properties and methods for this sub-class.

Define a **Customer** class which has a one-to-many association **"owns"** with the class **BankAccount**. Define necessary properties and methods for this class, the methods include at least:

- **add_account(account)**: add account to this customer
- **print_accounts()**: print out all the accounts owned by this customer
- **get_total_balance()**: get the **total** balance of all the customer's bank accounts

5. (5 marks) Please answer your Q&A question.

< Paste your question here >

Your answer:

International Software Engineering Program
International College
King Mongkut's Institute of Technology Ladkrabang

13016214 Software Engineering Principle

Final Examination

Tue 26th May 2020, 2:00 – 4:30 pm.

Instruction

1. This is an **opened book** exam.
2. There are **5** questions, please **answer all** of them.

Questions:

1. (5 marks) Draw **a use case diagram** for an on-line shopping system. Then draw **one sequence diagrams** for "place an order" use case for *its basic* course of event.

2. (5 marks) Draw a class diagram to model a file management system of a PC. The system is operated by a file manager on hard drives and/or removable drives. A removable drive can be a USB thumb drive or removable DVD drive.

A hard drive and a removable drive stores files and their tree structures containing directories, sub-directories, and files respectively. A directory can store sub-directories and these sub-directories can further store other sub-directories, and this can go on and on.

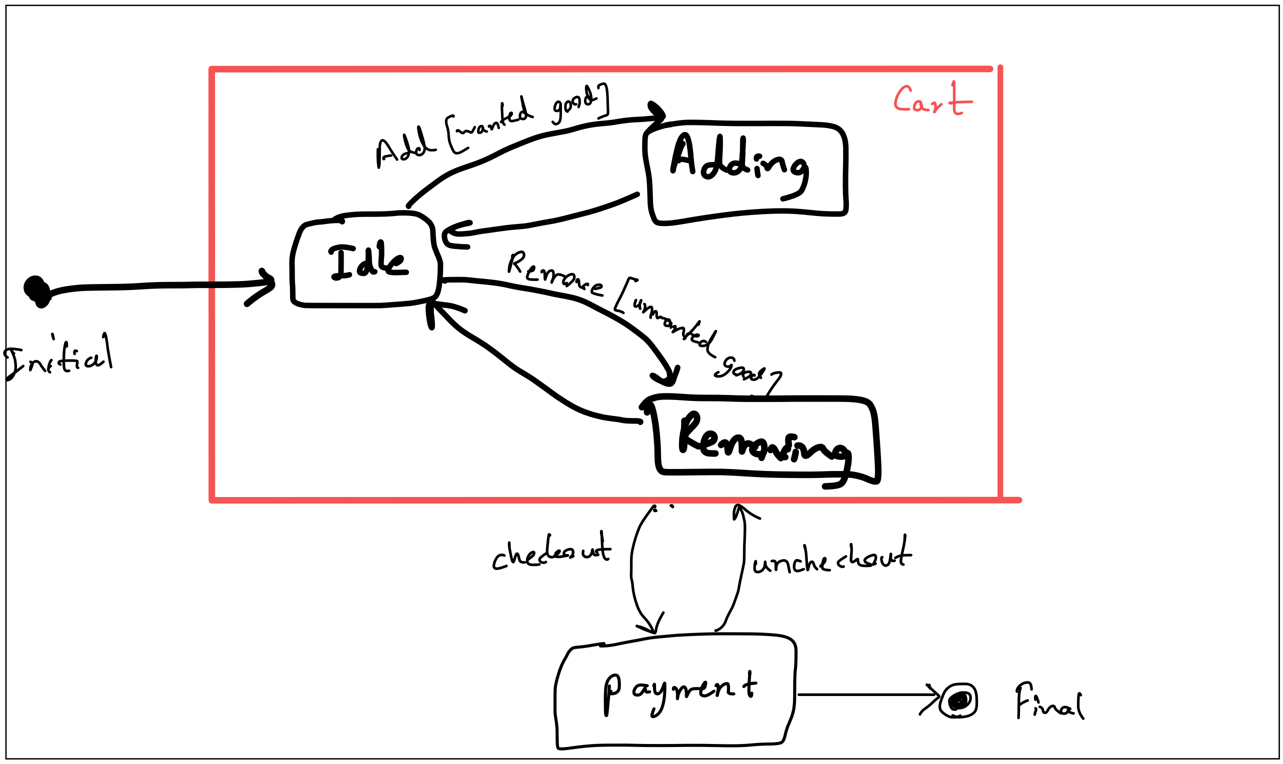
Shared properties between a hard drive and a removable drive are an icon image, size of used memory, and size of remaining memory of the drive.

Properties of a directory/sub-directory are an icon image, (memory) size, create date, and the numbers of items (files and subdirectories) it contains.

Properties of a file are an icon image, (memory) size, create date, update date, and the file owner.

The user can open/rename/format a hard drive. The user can open/rename/format/eject a removable drive. The use can open/rename/copy/delete files, directories, and sub-directories.

3. (5 marks) Draw **a statechart diagram** which describes all states and their transitions of a shopping cart. For an on-line shopping system, a shopping cart is where a customer put in the good he/she wants to buy temporally by keeping adding more good or removing some of them until no more good is wanted, then he/she will check out for payment and the cart will be no longer needed.



4. (5 marks) Please develop a Python program according to the following class diagram.

Define a **BankAccount** class which has the following **properties** and **methods**:

- **Properties**
  - **bank_name**: the name [illegible] ho creates the account
  - **acc_name**: the name [illegible] the account owner
  - **acc_id**: the account id
  - **balance**: the current b[illegible] count
  - **transaction_history**: transaction history of the account (stored in a list)
- **Methods**
  - **get_balance()**: get the current balance
  - **print_statement()**: pri[illegible]t according to **transaction_history**

Customer
BankAccount
SavingAccount
OverDrawnAccount

Define a **SavingAccount** a subclass of **Bank**[illegible] has additional **methods**:

- **deposit(money,person,date)**: deposit **money** by **person** to the account on **date**
- **withdraw(money,person,date)**: withdraw **money** by **person** from the account on **date**

Define a **OverDrawnAccount** as a sub-cla[illegible]**unt** class. An overdrawn account is different from a saving account very little, in that [illegible] cannot have a negative balance whilst an overdrawn account can have a negative balance but it does not exceed **the over drawn limit**. Please define necessary properties and methods for this sub-class.

Define a **Customer** class which has a one-to-many association **“owns”** with the class **BankAccount**. Define necessary properties and methods for this class, the methods include at least:

- **add_account(account)**: add account to this customer
- **print_accounts()**: print out all accounts this customer owns

5. (5 marks) Please answer your Q&A question.

< Paste your question here >
Your answer: